Project - Submission Sheet

Software Testing - Input Space Partitioning

Names: Sahana Vishwanath Karane
Group: Group 4

Software Testing - Summer 2024 - June 12, 2025 - Sibylle Schupp / Daniel Rashedi

Task 2 - Input Space Partitioning

a) Explain why the selected methods are suitable for the input domain model approach (1-2 sentences each)

Answer:

Selected function:
`http_status_to_exit_status(http_status: int, follow=False) -> ExitStatus`

- **Well-defined input categories:** The function takes a numeric status code and a boolean flag, making it easy to define input partitions based on HTTP status code ranges and redirect behavior.
- **Distinct behavioral branches:** Its output depends on whether the code is in the 2xx, 3xx, 4xx, or 5xx range, which maps clearly to equivalence classes for ISP testing.

b) Identify the input domain of your methods (2-3 sentences each)

Answer:

- The input domain of the `http_status_to_exit_status` function consists of two parameters: an integer `http_status` (typically in the range 100 to 599), and a boolean flag `follow`.
- The function uses conditional logic to map these status codes to different `ExitStatus` categories (e.g., 2xx, 3xx, 4xx, 5xx), with the behavior influenced by the value of `follow`.
- This input domain allows for clear partitioning based on both status code ranges and the presence or absence of the redirect-following flag.

c) Similar to Homework 02, gather the results of the following sub-tasks:

1) Identify reasonable characteristics of the possible input data
2) Derive sets of equivalence classes from these characteristics (Note: At least one of your partitions must consist of a number of blocks ≥ 3. Also make sure that you have enough characteristics and corresponding blocks to derive 12 distinct tests from them later on.)

Answer:

(a) Identify reasonable characteristics of the possible input data:
Analyzed the inputs to the function `http_status_to_exit_status(http_status: int, follow=False)`. Based on the return behavior and branching logic, identified the following characteristics:

- **HTTP status code range** – Whether the status code falls into the informational/success category (100–299), redirection (300–399), or client/server errors (400–599).

- **Redirect follow flag** – Whether the `follow` parameter is set to `True` or `False`, which influences how 3xx codes are handled.
- **Edge and special status values** – Whether the status code is a typical value (e.g., 200, 302), a boundary value (e.g., 299, 399, 599), or an invalid/out-of-range code (e.g., 600+).

(b) Derive sets of equivalence classes from these characteristics:

Characteristic	**Block / Eq-Class 1**	**Block / Eq-Class 2**	**Block / Eq-Class 3**
q_1 = HTTP status code range	A1: 100–299 (Informational/Success)	A2: 300–399 (Redirect)	A3: 400–599 (Client/Server Error)
q_2 = Follow flag	B1: `follow = False`	B2: `follow = True`	—
q_3 = Specific status edge values	C1: Typical (200, 302, 404)	C2: Boundary (299, 399, 599)	C3: Invalid/Out of Range (600+)

Table 1: Equivalence Classes Based on Input Characteristics

Note: The characteristic q_2 (Follow flag) has only two logical states: `True` and `False`. Since it is a boolean input, it inherently consists of only two possible values, and there is no meaningful or distinct third equivalence class. Adding another block would violate the principle of mutual exclusivity and completeness in input space partitioning.

d) Can you identify any non-valid combinations of blocks from your characteristics? (1-2 sentences each)

Answer:

Non-Valid Combinations of Blocks

- **A1 (Success codes: 100–299) + C3 (Invalid codes: 600+)**
  A1 contains valid HTTP success/informational codes, while C3 includes values that are out-of-range. A status code cannot be both valid (e.g., 200) and invalid (e.g., 600), making this combination logically inconsistent.
- **A2 (Redirect codes: 300–399) + C3 (Invalid codes: 600+)**
  A2 defines valid redirect status codes, such as 301 or 302. These cannot overlap with C3 values like 600, which fall outside the HTTP specification.
- **A3 (Client/server error codes: 400–599) + C3 (Invalid codes: 600+)**
  A3 includes defined client and server error codes like 404 and 503. Since C3 contains undefined or invalid codes, combining the two results in contradiction.
- **Any A-block (A1–A3) + C3 (Invalid codes: 600+)**
  In general, C3 values lie outside the valid HTTP range defined in A1–A3. Therefore, any combination pairing a valid category with an out-of-spec value is invalid.

These combinations may lead to invalid or unintended behavior during path generation and should be avoided or handled explicitly in implementation or testing.

e) Derive (at least) one representative value for each equivalence class. Which strategy did you use for the value selection?

Answer:

(e) Representative Values and Selection Strategy

Representative values were chosen for each equivalence class using a combination of *typical values*, *boundary values*, and *valid edge inputs*. These help in systematically exploring the behavior of the `http_status_to_exit_status` function across different partitions — including common HTTP codes, limit cases within each status code category, and unexpected out-of-range values.

Characteristic	**Block / Eq-Class**	**Representative Value**
HTTP status code range	A1: 100–299 (Success)	`200`
	A2: 300–399 (Redirect)	`302`
	A3: 400–599 (Error)	`404`
Follow flag	B1: follow = False	`False`
	B2: follow = True	`True`
Specific/edge status values	C1: Typical values	`200, 302, 404`
	C2: Boundary values	`299, 399, 599`
	C3: Invalid/out-of-range	`600`

Table 2: Representative Values Derived from ISP Equivalence Classes

f) Combine the selected values from the partitions of each input argument to complete test vectors using `hypothesis`. Document only the excerpt of your code responsible for generation of value combinations.

Answer:

(f) Test Vector Combinations

We combine the representative values across characteristics to form test vectors. Below is a listing of selected test inputs used for testing:

Test Case	HTTP Status	Follow	Block Mapping (with labels)
TC1	200	False	A1 – B1 – C1 (Success – NoFollow – Typical)
TC2	299	True	A1 – B2 – C2 (Success – Follow – Boundary)
TC3	302	False	A2 – B1 – C1 (Redirect – NoFollow – Typical)
TC4	399	True	A2 – B2 – C2 (Redirect – Follow – Boundary)
TC5	404	False	A3 – B1 – C1 (Error – NoFollow – Typical)
TC6	600	True	— – B2 – C3 (Invalid – Follow – OutOfRange)

Table 3: 6 Test Vector Combinations with Block Mapping and Labels

g) Create Python tests for your method, which trigger test runs with the selected test vectors/blocks, and assert each individual output. For the submission, merge all these tests into one `*.py` file.

Answer:

```python
import pytest
from hypothesis import given, strategies as st, settings
from httpie.status import http_status_to_exit_status, ExitStatus  # Adjust import path if needed

# ISP Test Cases: 6 combinations

@settings(max_examples=6)  # Run exactly 6 combinations
@given(
    http_status=st.sampled_from([
        200,  # A1 - C1: Success typical
        299,  # A1 - C2: Success boundary
        302,  # A2 - C1: Redirect typical
        399,  # A2 - C2: Redirect boundary
        404,  # A3 - C1: Client error
        600   # C3: Out-of-range
    ]),
    follow=st.sampled_from([False, True])  # B1 and B2
)
def test_http_status_to_exit_status_limited(http_status, follow):
    """ISP: Combined test of http_status (q1, q3) and follow flag (q2)"""

    print(f"\nTesting with: http_status = {http_status}, follow = {follow}")

    result = http_status_to_exit_status(http_status, follow)
```

```python

    # === Assertions ===
    if 300 <= http_status <= 399 and not follow:
        assert result == ExitStatus.ERROR_HTTP_3XX
    elif 400 <= http_status <= 499:
        assert result == ExitStatus.ERROR_HTTP_4XX
    elif 500 <= http_status <= 599:
        assert result == ExitStatus.ERROR_HTTP_5XX
    else:
        assert result == ExitStatus.SUCCESS

# Annotations for ISP Blocks (for comments/documentation):
# You can annotate each test vector combination like this:

# Test: A2 (300  399  ) + B1 (follow = False) + C1 (typical)
# Input: http_status = 302, follow = False

# Testing with: http_status = 404, follow = True
#     A3 (Client Error) + B2 (follow = True) + C1

# @settings(max_examples=6) decorator added to ensure
# Hypothesis runs 6 combinations only (for controlled execution)

# In your @given decorator, you are combining:
# - 6 possible values for http_status (from  q   and  q  )
#         Includes typical (C1), boundary (C2), and invalid (C3)
# - 2 possible values for follow (False, True)
#         Corresponds to B1 and B2

# Hypothesis will dynamically combine these while staying within the 6-test limit.
```

AI Review Task

Apply viewpoint analysis to your LLM's generated test cases. Add any code relevant for your analysis to the answer:

Answer:

[Group: Your LLM]
[Group: Your Prompts] ...
[Group: Your Viewpoint Analysis]